

Extended A/B Testing Lab: Pricing Optimization for FarmBurg

Project Overview

Brian is a Product Manager at FarmBurg, a farming simulation social network game. He wants to optimize pricing for a microtransaction upgrade package using rigorous A/B testing methodology. This extended lab will guide you through comprehensive statistical analysis beyond basic hypothesis testing.

Dataset Description

Column	Type	Description
user_id	String	Unique identifier for each visitor
group	Category	'A' (\$0.99), 'B' (\$1.99), or 'C' (\$4
is_purchase	Binary	'Yes' if purchase made, 'No' othe

Additional data available: time_on_site, device_type, user_age_group, prior_purchase_count

Tasks: Comprehensive Analysis Workflow

Task 1: Data Loading & Initial Exploration

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

# Load data
abdata = pd.read_csv('clicks.csv')

# Extended inspection
print(f"Total records: {len(abdata)}")
print(f"Null values:\n{abdata.isnull().sum()}")
print(f"Group distribution:\n{abdata['group'].value_counts()}")
print(f"Purchase distribution:\n{abdata['is_purchase'].value_counts()}")
```

Task 2: Contingency Table & Chi-Square Test

```
# Create contingency table
Xtab = pd.crosstab(abdata.group, abdata.is_purchase)
```

```

print(Xtab)

# Chi-square test
from scipy.stats import chi2_contingency
chi2, pval, dof, expected = chi2_contingency(Xtab)
print(f"Chi-square statistic: {chi2:.4f}")
print(f"P-value: {pval:.6e}")
print(f"Degrees of freedom: {dof}")

# Interpretation
alpha = 0.05
if pval < alpha:
    print("✓ Significant difference exists between groups")
else:
    print("✗ No significant difference detected")

```

Task 3: Purchase Rate Calculation per Group

```

purchase_rates = {}
for group in ['A', 'B', 'C']:
    group_data = abdata[abdata['group'] == group]
    total = len(group_data)
    purchases = np.sum(group_data['is_purchase'] == 'Yes')
    rate = purchases / total * 100
    purchase_rates[group] = {'total': total, 'purchases': purchases, 'rate':
rate}

for g, data in purchase_rates.items():
    print(f"Group {g}: {data['purchases']}/{data['total']} =
{data['rate']:.2f}%")

```

Task 4: Revenue Target Analysis

```

target_revenue = 1000
weekly_visitors = len(abdata)
prices = {'A': 0.99, 'B': 1.99, 'C': 4.99}

required_sales_and_rates = {}
for g, price in prices.items():
    sales_needed = target_revenue / price
    purchase_rate_needed = (sales_needed / weekly_visitors) * 100
    required_sales_and_rates[g] = {
        'price': price,
        'sales_needed': sales_needed,
        'rate_needed_pct': purchase_rate_needed
    }

for g, info in required_sales_and_rates.items():

```

```
print(f"Group {g} ({info['price']}): Need {info['sales_needed']:.0f}
sales ({info['rate_needed_pct']:.2f}%)")
```

Task 5: Binomial Tests Against Targets

```
from scipy.stats import binom_test

results = {}
for g in ['A', 'B', 'C']:
    samp_size = purchase_rates[g]['total']
    sales = purchase_rates[g]['purchases']
    target_rate = required_sales_and_rates[g]['rate_needed_pct'] / 100

    p_value = binom_test(sales, n=samp_size, p=target_rate,
alternative='greater')
    results[g] = {
        'sample_size': samp_size,
        'sales': sales,
        'observed_rate': purchase_rates[g]['rate'],
        'target_rate': target_rate * 100,
        'p_value': p_value,
        'significant': p_value < 0.05
    }

for g, r in results.items():
    sig_mark = "✓ SIGNIFICANT" if r['significant'] else "X Not significant"
    print(f"\nGroup {g} ({required_sales_and_rates[g]['price']}):")
    print(f"  Observed: {r['observed_rate']:.2f}% | Target:
{r['target_rate']:.2f}%")
    print(f"  p-value: {r['p_value']:.6f} | {sig_mark}")
```

Task 6: Confidence Intervals

```
from statsmodels.stats.proportion import proportion_confint

print("\nConfidence Intervals (95%):")
for g in ['A', 'B', 'C']:
    ci_low, ci_high = proportion_confint(
        count=purchase_rates[g]['purchases'],
        nobs=purchase_rates[g]['total'],
        alpha=0.05,
        method='normal'
    )
    print(f"Group {g}: [{ci_low*100:.2f}%, {ci_high*100:.2f}%]")
```

Task 7: Expected Weekly Revenue

```
print("\nProjected Weekly Revenue:")
for g in ['A', 'B', 'C']:
    exp_revenue = (purchase_rates[g]['purchases'] / purchase_rates[g]
['total']) * weekly_visitors * required_sales_and_rates[g]['price']
    meets_target = "✓ MEETS $1000 TARGET" if exp_revenue >= 1000 else "X
Below target"
    print(f"Group {g}: ${exp_revenue:.2f}/week | {meets_target}")
```

Task 8: Effect Size Calculation

```
print("\nEffect Sizes (Cohen's h):")
for g1 in ['A', 'B', 'C']:
    for g2 in ['A', 'B', 'C']:
        if g1 != g2:
            p1 = purchase_rates[g1]['rate'] / 100
            p2 = purchase_rates[g2]['rate'] / 100
            h = 2 * (np.arcsin(np.sqrt(p1)) - np.arcsin(np.sqrt(p2)))
            effect = "large" if abs(h) > 0.8 else "medium" if abs(h) > 0.5
else "small" if abs(h) > 0.2 else "negligible"
            print(f"{g1} vs {g2}: h={h:.3f} ({effect})")
```

Task 9: Statistical Power Analysis

```
from statsmodels.stats.power import GofChisquarePower, NormalIndPower

print("\nStatistical Power Analysis:")
power_analysis = GofChisquarePower()
# For chi-square with df=2, alpha=0.05
power = power_analysis.solve_power(effect_size=0.3, nobs1=len(abdata)/3,
alpha=0.05, k_rows=2, k_cols=2)
print(f"Test power with current sample: {power*100:.1f}%")
```

Task 10: Decision Framework

```
scoring = []
for g in ['A', 'B', 'C']:
    score = 0
    # Meets revenue target (+30)
    if results[g]['significant']:
        score += 30
    # Highest observed rate (+25)
    if g == max(purchase_rates, key=lambda x: purchase_rates[x]['rate']):
        score += 25
    # Reasonable price point (+20)
    if required_sales_and_rates[g]['rate_needed_pct'] < 10:
        score += 20
```

```
# Lower price accessibility (+15)
if required_sales_and_rates[g]['price'] <= 1.99:
    score += 15

# Profit margin consideration (+10)
if required_sales_and_rates[g]['price'] >= 1.99:
    score += 10

scoring.append({'group': g, 'score': score})

best_option = max(scoring, key=lambda x: x['score'])
print(f"\n== RECOMMENDATION ==")
print(f"Best Price Point: ${required_sales_and_rates[best_option['group']]
['price']} (Group {best_option['group']})")
print(f"Decision Score: {best_option['score']}/100")
```



Top 10 Insights from A/B Testing Analysis

Insight #1: Statistical Significance ≠ Business Significance

A statistically significant result (low p-value) doesn't automatically mean the change improves business outcomes. In this lab, Group A had the highest purchase rate statistically, but that doesn't necessarily generate optimal revenue. Always evaluate against specific business goals (like \$1000/week revenue target) rather than just p-values.

Insight #2: Multiple Testing Problem

When testing multiple hypotheses simultaneously (comparing 3 price points), you inflate the family-wise error rate. With 3 groups and 2 pairwise comparisons each, you need Bonferroni correction: $\alpha_{\text{corrected}} = 0.05/3 \approx 0.017$. Otherwise, you risk false positives. Consider using ANOVA followed by post-hoc tests instead of multiple independent tests.

Insight #3: Confidence Intervals Tell a Richer Story

While p-values give binary significance, confidence intervals show the precision and practical range of effects. A narrow CI around $12\% \pm 2\%$ gives more actionable insight than " $p < 0.05$." If CIs overlap substantially between groups, differences may not be practically meaningful despite statistical significance.

Insight #4: Sample Size Affects Detection Power

Small samples can miss real effects (Type II errors). The power analysis reveals whether your test design can detect meaningful differences. With current data, if power is below 80%, you risk concluding "no difference" when one exists. Plan sample sizes prospectively using power calculations before running tests.

Insight #5: Price Elasticity Reveals Optimal Pricing

The relationship between price and conversion rate is rarely linear. Group C (\$4.99) had lower purchase rate but potentially higher revenue per customer. Calculate the elasticity: $\% \Delta Q / \% \Delta P$. The optimal price maximizes (price $\times$ conversion_rate $\times$ volume), not just conversion rate alone.

Insight #6: Segmentation Uncovers Hidden Patterns

Aggregated results can mask important subgroup differences. Younger users may prefer \$4.99 premium features while casual players convert better at \$0.99. Segment by: device type, user tenure, geographic region, prior spending history. What works for your average user may fail for valuable customer segments.

Insight #7: Long-Term Value Trumps Initial Conversion

Short-term conversion optimization can reduce Customer Lifetime Value (CLV). High-price buyers might spend more overall if satisfied. Track cohorts over 30-90 days: repeat purchases, churn rates, referral activity. A 20% drop in initial conversion with 3× CLV is superior to high initial sales with zero retention.

Insight #8: Selection Bias Can Skew Results

If group assignment isn't truly random, results become invalid. Check balance: demographic distributions, baseline behavior metrics across groups before treatment. Self-selection bias occurs if users opt into groups rather than being randomly assigned. Randomization checks should confirm no pre-treatment differences.

Insight #9: Novelty and Learning Effects Distort Early Results

Initial spikes in conversion may reflect curiosity, not sustained preference. Run tests long enough to capture full cycle (typically 2 weeks for weekly cycles). Exclude first-day data if novelty effects are suspected. Seasonal patterns (weekends, holidays) can confound results—ensure test period represents normal operations.

Insight #10: Cost of Wrong Decisions Outweighs Test Rigor

Deploying wrong pricing costs more than running longer tests. Create decision frameworks weighing: statistical evidence strength, implementation cost, reversibility, competitive landscape impact. Sometimes accepting 80% certainty with faster deployment beats 95% certainty with delayed market entry. Balance rigor with business velocity needs.



Final Recommendation Template

Pricing Decision Report

****Recommended Price Point:**** \$4.99 (Group C)

****Rationale:****

- ✓ Statistically significantly exceeds revenue threshold (p = 0.031)
- ✓ Projects \$1,127/week revenue (127% of target)
- ✓ Highest profit margin per unit sold
- ⚠ Requires careful communication to avoid customer pushback

****Risk Mitigation:****

1. Implement gradual rollout to 10% of users first
2. Monitor customer satisfaction surveys weekly
3. Prepare discount codes for customers who churn
4. Have \$2.99 fallback option ready

```
**Success Metrics (30-day tracking):**  
- Primary: Weekly revenue  $\geq$  $1000  
- Secondary: Churn rate  $<$  5%, NPS score  $>$  30  
- Guardrail: Support tickets related to pricing  $\leq$  baseline + 10%
```



Advanced Extensions for Future Analysis

1. **Bayesian A/B Testing** — Incorporate prior beliefs and update probabilities continuously
2. **Multi-Armed Bandit Algorithms** — Dynamically allocate traffic to winning variants during the test
3. **Causal Inference Methods** — Propensity score matching for observational data analysis
4. **Machine Learning Models** — Predict individual conversion probability based on user features
5. **Revenue Per Visitor Optimization** — Factor in secondary purchases, referrals, and engagement metrics

This extended framework transforms basic hypothesis testing into a comprehensive business decision-making pipeline suitable for production environments.